

INSTITUTO FEDERAL DA PARAÍBA – IFPB

DISCIPLINA: Sistemas Embarcados

PROFESSOR: Alexandre Sales Vasconcelos

EQUIPE: Bráulio Matheus Brito Barbosa

CURSO: Engenharia de Computação

Entrega: Projeto Final: Mesa Labirinto – Fase 3

1. Introdução

Este projeto foi desenvolvido para a disciplina de **Sistemas Embarcados** com o objetivo de aplicar conceitos de programação embarcada, controle de periféricos e sistemas operacionais de tempo real utilizando a plataforma **ESP32-S3** e o framework **ESP-IDF**.

A proposta consiste na implementação de uma **Mesa Labirinto Inteligente**, na qual a inclinação da plataforma é controlada por dois servomotores acionados por um joystick analógico. Além disso, um sensor **MPU6050** realiza o monitoramento da orientação da mesa, permitindo acompanhar sua posição durante a operação.

O desenvolvimento foi realizado de forma incremental ao longo das três fases da disciplina, culminando em um sistema organizado em módulos e estruturado com tarefas concorrentes utilizando o **FreeRTOS**.

2. Objetivos

Objetivo Geral

Desenvolver um sistema embarcado capaz de controlar uma mesa labirinto de dois eixos utilizando o ESP32-S3, integrando sensores, atuadores e uma arquitetura multitarefa baseada em FreeRTOS.

Objetivos Específicos

- Implementar o controle de dois servomotores responsáveis pela inclinação da mesa;
- Realizar a leitura contínua do joystick analógico através do conversor ADC;
- Monitorar a orientação da mesa utilizando o sensor MPU6050;
- Organizar o software em módulos independentes;
- Implementar uma arquitetura multitarefa utilizando o FreeRTOS;
- Desenvolver um sistema escalável para futuras expansões.

3. Fundamentação Teórica

Os sistemas embarcados são sistemas computacionais dedicados à execução de funções específicas, normalmente integrando sensores, atuadores e dispositivos de processamento em tempo real.

Neste projeto foi utilizada a plataforma ESP32-S3, um microcontrolador de alto desempenho desenvolvido pela Espressif, que possui recursos suficientes para aplicações de automação, robótica e Internet das Coisas.

O controle da inclinação da mesa é realizado por servomotores do tipo SG90, cuja posição é definida através de sinais PWM (Pulse Width Modulation). A posição desejada é obtida a partir das leituras analógicas do joystick, realizadas pelo conversor ADC interno do ESP32.

Para monitorar a orientação da plataforma foi empregado o sensor MPU6050, composto por acelerômetro e giroscópio de três eixos, cuja comunicação ocorre através do barramento I²C.

A execução simultânea das diferentes funcionalidades do sistema foi realizada utilizando o FreeRTOS, permitindo que múltiplas tarefas fossem executadas concorrentemente.

4. Materiais Utilizados

Para o desenvolvimento do projeto foram utilizados os seguintes componentes:

- ESP32-S3 DevKitC-1;
- Dois servomotores SG90;
- Joystick analógico;
- Sensor MPU6050;
- Capacitores eletrolíticos para estabilização da alimentação dos servomotores;
- Capacitor cerâmico para desacoplamento da alimentação do sensor MPU6050;
- Protoboard;
- Jumpers;
- Fonte de alimentação externa para os servomotores.

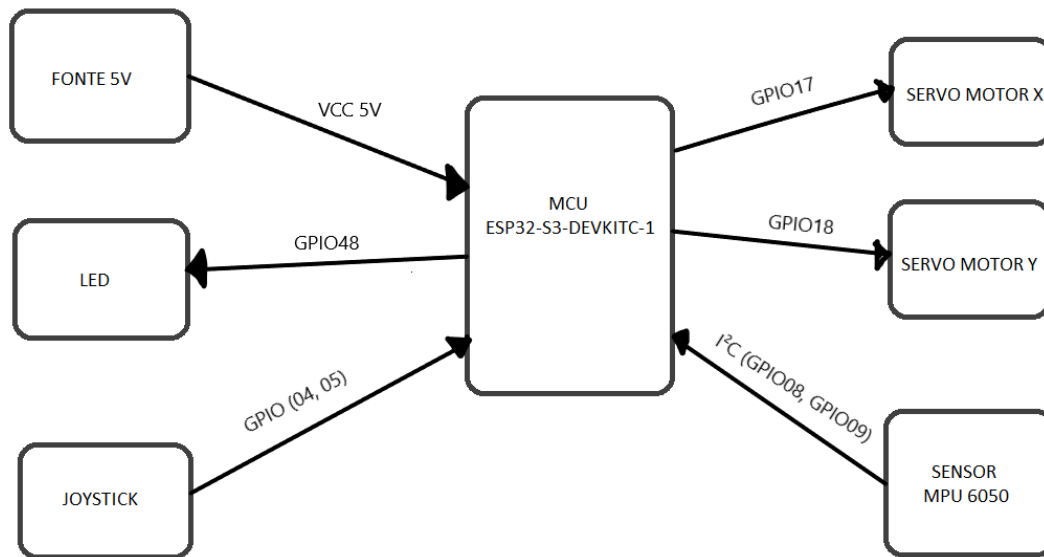


DIAGRAMA DE BLOCOS - PROJETO FINAL: MESA LABIRINTO

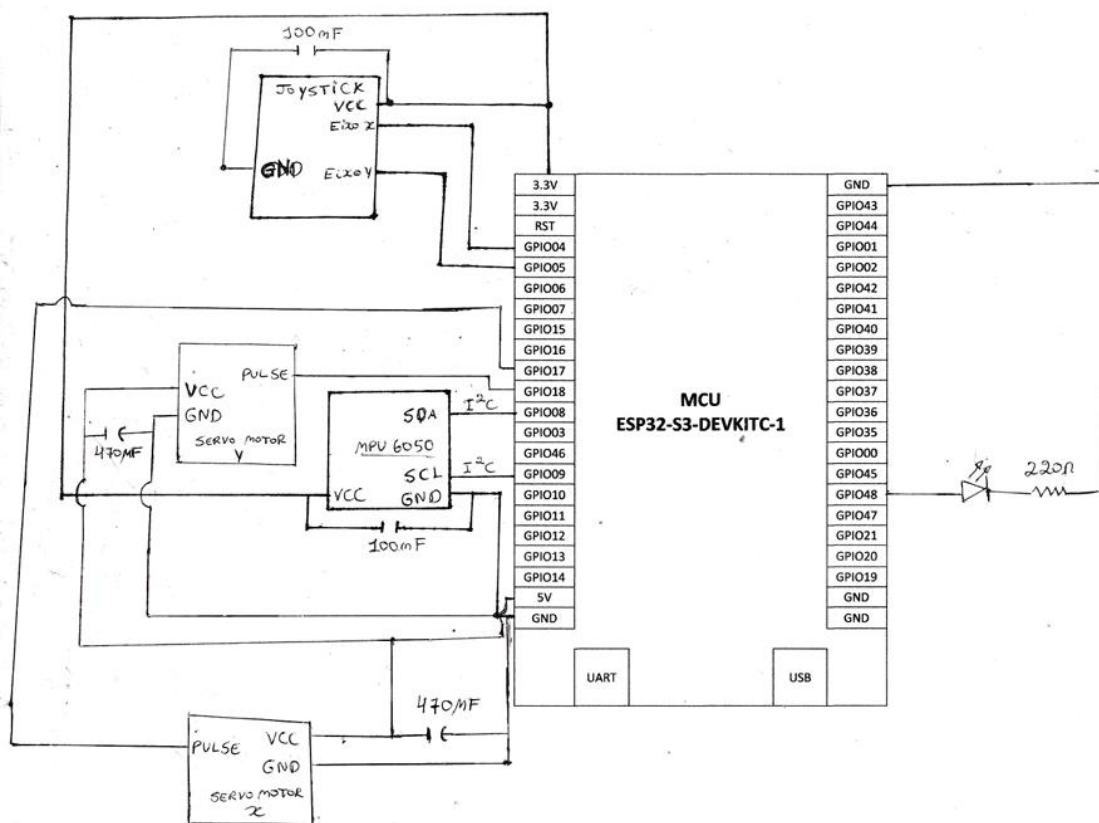


DIAGRAMA ESQUEMÁTICO- PROJETO FINAL: MESA LABIRINTO

Também foram utilizados os seguintes recursos de software:

- ESP-IDF;
- Linguagem C;
- FreeRTOS;
- Driver ADC;
- Driver GPIO;
- Driver LEDC (PWM);
- Driver I²C.

5. Desenvolvimento

O sistema foi dividido em módulos independentes, facilitando a organização do código e sua manutenção.

Durante a inicialização do sistema são executadas as seguintes etapas:

- Configuração do barramento I²C;
- Inicialização do sensor MPU6050;
- Configuração do PWM dos servomotores;
- Configuração do ADC para leitura do joystick;
- Criação das tarefas do FreeRTOS.

Após essa etapa, o sistema entra em operação contínua.

O joystick é monitorado constantemente, sendo suas posições convertidas em ângulos para os servomotores responsáveis pelos eixos X e Y da mesa.

Simultaneamente, o MPU6050 realiza leituras da orientação da plataforma, permitindo acompanhar sua inclinação durante o funcionamento.

Para garantir maior estabilidade elétrica do sistema, foram empregados capacitores em pontos estratégicos do circuito. Os servomotores apresentam picos de corrente durante partidas e mudanças rápidas de posição, podendo causar quedas momentâneas de tensão na alimentação. Para minimizar esses efeitos, foi utilizado um **capacitor eletrolítico de filtragem** conectado entre a alimentação de 5 V e o GND dos

servomotores, atuando como um reservatório temporário de energia e reduzindo oscilações de tensão.

Além disso, foi empregado um **capacitor cerâmico de desacoplamento** próximo ao sensor MPU6050. Esse capacitor tem a função de filtrar ruídos de alta frequência presentes na alimentação do circuito, contribuindo para maior estabilidade das leituras obtidas pelo barramento I²C e reduzindo interferências eletromagnéticas.

Toda essa lógica ocorre de forma concorrente através das tarefas do FreeRTOS.

5.1 Estabilização da Alimentação

Durante o desenvolvimento do circuito, foi necessária a utilização de capacitores para garantir a estabilidade elétrica do sistema. Como os servomotores podem consumir correntes elevadas durante acelerações e mudanças bruscas de posição, um capacitor eletrolítico foi conectado entre a alimentação de 5 V e o GND, funcionando como um elemento de armazenamento temporário de carga e reduzindo quedas instantâneas de tensão.

Também foi utilizado um capacitor cerâmico de desacoplamento próximo ao sensor MPU6050. Esse componente tem como finalidade filtrar ruídos de alta frequência provenientes da alimentação, contribuindo para leituras mais estáveis do acelerômetro e giroscópio, além de melhorar a confiabilidade da comunicação pelo barramento I²C.

Esses componentes, embora simples, desempenham um papel importante na robustez do circuito, reduzindo interferências elétricas e aumentando a estabilidade geral do sistema durante sua operação.

6. Arquitetura do Software

O projeto foi estruturado de forma modular.

A organização dos arquivos é composta pelos seguintes módulos:

- main.c
- joystick.c
- joystick.h
- servo.c
- servo.h
- mpu6050.c
- mpu6050.h
- i2c.c
- i2c.h
- freertos_tasks.c

Essa divisão permite separar claramente as responsabilidades de cada componente do sistema, facilitando futuras modificações.

7. Funcionamento do Sistema

O funcionamento ocorre conforme o seguinte fluxo:

1. Inicialização do ESP32-S3.
2. Configuração dos periféricos.
3. Inicialização do barramento I²C.
4. Inicialização do MPU6050.
5. Configuração dos servomotores.
6. Leitura contínua do joystick.
7. Conversão da posição do joystick em ângulos dos servos.
8. Atualização da posição da mesa.
9. Leitura contínua do sensor MPU6050.
10. Execução simultânea das tarefas pelo FreeRTOS.

Essa arquitetura permite que o controle dos servos e a aquisição dos sensores ocorram de maneira independente.

8. Desenvolvimento por Etapas

Fase 1

Na primeira etapa foi realizada a configuração do ambiente de desenvolvimento utilizando o ESP-IDF, bem como a implementação do controle básico dos servomotores e da leitura do joystick analógico.

Também foi criada a estrutura inicial do projeto.

Fase 2

Na segunda etapa ocorreu a integração do sensor MPU6050 utilizando comunicação I²C.

Além disso, foi realizada a reorganização do software utilizando o FreeRTOS, separando as funcionalidades em tarefas independentes.

O código também passou a ser organizado em módulos.

Fase 3

Na etapa final foi realizada a integração completa entre joystick, servomotores e MPU6050.

Foram efetuados ajustes de estabilidade e desempenho, além da validação do funcionamento integrado do sistema.

9. Resultados Obtidos

O projeto apresentou funcionamento conforme o esperado.

Foi possível:

- Controlar a inclinação da mesa através do joystick;
- Acionar os servomotores de forma suave;
- Realizar a leitura contínua da orientação utilizando o MPU6050;
- Executar múltiplas tarefas simultaneamente utilizando o FreeRTOS;
- Garantir maior estabilidade da alimentação dos servomotores por meio do uso de capacitores de filtragem;

- Reduzir ruídos elétricos na alimentação do sensor MPU6050 utilizando capacitor de desacoplamento;
- Organizar o software de forma modular e escalável.

A utilização dos capacitores contribuiu para reduzir oscilações na alimentação, aumentando a confiabilidade do sistema durante o funcionamento simultâneo dos periféricos.

9.1 Discussão dos Resultados

Durante os testes práticos do sistema foram observadas algumas limitações inerentes aos componentes utilizados e ao ambiente de desenvolvimento, as quais não comprometeram o funcionamento do projeto, mas influenciaram seu comportamento durante a demonstração.

Um dos aspectos observados foi uma leve trepidação da mesa ocasionada pelos servomotores. Esse comportamento ocorre devido ao mecanismo interno de autocorreção de posição dos servos, que ajustam continuamente seu eixo para manter o ângulo determinado pelo sinal PWM. Pequenas variações na leitura do joystick, tolerâncias mecânicas dos servomotores e oscilações naturais do sistema fazem com que os motores realizem correções frequentes, resultando em pequenas vibrações da plataforma. Apesar desse comportamento, o controle da mesa permaneceu funcional e compatível com os objetivos propostos pelo projeto.

Outra dificuldade encontrada esteve relacionada à demonstração da execução das tarefas do FreeRTOS utilizando a ferramenta de visualização de tarefas disponível na IDE. Devido a uma incompatibilidade entre a versão utilizada da IDE e o suporte ao ESP32-S3, não foi possível exibir corretamente as tarefas em execução por meio desse recurso gráfico durante a apresentação do projeto.

Como alternativa para validar o funcionamento da arquitetura multitarefa, foi implementada a exibição das informações diretamente pela saída serial. Cada tarefa passou a enviar mensagens contendo os dados processados, como as leituras do joystick, do sensor MPU6050 e os comandos enviados aos servomotores. Dessa forma, foi possível demonstrar de maneira clara que as tarefas estavam sendo executadas de forma concorrente e independente, comprovando o correto funcionamento da arquitetura baseada em FreeRTOS, mesmo sem o auxílio da ferramenta gráfica da IDE.

10. Possíveis Melhorias

Embora o sistema tenha atingido seus objetivos, diversas melhorias podem ser implementadas futuramente, entre elas:

- Controle automático da esfera utilizando controlador PID;
- Exibição das informações em display OLED;
- Comunicação Wi-Fi para monitoramento remoto;
- Registro das medições em cartão microSD;
- Interface Web para configuração e monitoramento;
- Controle por aplicativo móvel.

Essas funcionalidades ampliariam significativamente as possibilidades de utilização do sistema.

11. Conclusão

O desenvolvimento da Mesa Labirinto Inteligente permitiu aplicar, de forma prática, diversos conceitos fundamentais de Sistemas Embarcados, incluindo programação em linguagem C, utilização do ESP32-S3, comunicação com sensores e atuadores, geração de sinais PWM, conversão analógico-digital e comunicação I²C.

A utilização do FreeRTOS possibilitou a implementação de uma arquitetura multitarefa organizada, tornando o sistema mais modular, eficiente e de fácil manutenção.

Os resultados obtidos demonstram que os objetivos propostos foram alcançados com sucesso, fornecendo uma base sólida para futuras expansões, como a implementação de algoritmos de controle automático e interfaces de monitoramento remoto. O projeto também contribuiu para consolidar conhecimentos práticos relacionados ao desenvolvimento de sistemas embarcados modernos, evidenciando a importância da integração entre hardware e software na construção de soluções robustas e escaláveis.